

NOTE DE SYNTHÈSE EXECUTIVE

Reconciliation Paie-ment-Facture par IA

Système de matching automatique

pour le factoring

Préparé par : Joseph Gesnouin

Date : Avril 2026

Classification : Confidentiel

Méthode : Pair-programming avec Claude Code (Anthropic)

1. SYNTHÈSE EXECUTIVE

Ce document présente le retour d'expérience sur la conception et la réalisation d'un système complet de reconciliation paiement-facture par intelligence artificielle, construit en pair-programming avec Claude Code, l'agent IA d'Anthropic.

1.1 Objectif

Automatiser le rapprochement des paiements entrants avec les factures ouvertes dans un contexte de factoring international (40+ pays, 12 langues), en maximisant le taux de reconciliation automatique tout en minimisant les faux positifs.

1.2 Resultats obtenus

Taux d'automatisation	90.3% (objectif : 95%)
Precision	~99.9% (zero faux positif C1/C2)
Volume teste	5 270 factures, 4 724 paiements
Couverture	50 debiteurs, 40+ pays, 12 langues
Temps de traitement	57 paiements/seconde
Recommandation ML	77% de precision top-5 pour la revue humaine

1.3 Delai de realisation

6 jours de travail, 1 personne + Claude Code (agent IA)

Estimation classique pour un projet equivalent : 3-4 mois avec une equipe de 3-4 developpeurs. Le gain de temps est d'un facteur 30 a 80 selon les taches.

2. ARCHITECTURE TECHNIQUE

Le systeme repose sur une architecture a 6 couches avec principe d'early-exit : chaque paiement traverse les couches de la plus rapide (deterministe) a la plus complexe (IA generative), et sort du pipeline des qu'un match fiable est trouve.

2.1 Les 6 couches

Couche	Nom	Methode	Confiance
C0	Preprocessing	14 transforms (uppercase, OCR, IBAN, dates...)	—
C1	Exact Match	Reference, ISO20022, IBAN+montant, PO/BL	97-100%
C2	Regles Metier	Tolerances, subset sum, avoirs, acomptes	85-99%
C3	NLP / Fuzzy	8 algos fuzzy, OCR fix, NER, TF-IDF	75-92%
C4	Machine Learning Ensemble	LightGBM+XGBoost+RF, 42 features	80-95%
C5	LLM	Claude/GPT pour cas ambigus (optionnel)	Variable
C6	Revue Humaine	File priorisee + recommandations ML	100%

2.2 Distribution mesuree

Couche	Paielements	%
C1 Exact	2 689	57%
C2 Regles Metier	669	14%
C3 NLP/Fuzzy	253	5%
C4 Machine Learning	656	14%
C6 Revue Humaine	457	10%

2.3 Innovations techniques

- Correction OCR automatique (0<>O, 1<>I) dans le fuzzy matching
- Meet-in-the-middle DP pour le subset sum (jusqu'a 24 factures)
- 7 hypotheses de tolerance sur les sommes multi-factures
- Recommandations ML composites (ML 60% + heuristique 40%) avec explainability
- Debtor Behavior Profiler : apprentissage automatique des patterns par debiteur

3. METHODE DE TRAVAIL AVEC CLAUDE CODE

Le projet a été réalisé en pair-programming avec Claude Code, l'agent IA d'Anthropic. Ce n'est pas un simple assistant de code — c'est un ingénieur logiciel autonome capable de lire, comprendre, modifier et tester l'ensemble d'un codebase.

3.1 Repartition des rôles

Aspect	Joseph (humain)	Claude Code (IA)
Vision	Cahier des charges	—
Specs	En langage naturel	Traduit en architecture
Code	—	14 548 lignes Python
Tests	Validation fonctionnelle	127 tests unitaires
Debug	"Ça marche pas"	Diagnostic 5 root causes
UX	"C'est pas assez propre"	Redesign premium complet
Audit	"Optimise tout"	22 bugs, perf 2.5x
Docs	"Rajoute des README"	5 README + rapport + PDF

3.2 Exemples d'interactions réelles

Voici des échanges verbatim de la session de travail :

Joseph : *"je veux simuler plein de transactions"*

Claude Code a généré 500 factures, 400 paiements, 12 débiteurs sur 6 mois.

Joseph : *"C3 et C4 marchent pas du tout"*

Claude Code a diagnostiqué 5 root causes, corrigé C3 (0%→11%), C4 (0%→14%). Taux auto : 65%→93.6%.

Joseph : *"rajoute des fautes de frappe réalistes"*

Claude Code a créé 16 opérateurs de mutation (swap, 0→O, truncate...) en 3 niveaux de sévérité.

Joseph : *"le modèle ML affiche 0% partout"*

Claude Code a diagnostiqué que le training n'utilisait que des paiements avec ref. Fix : 3 sources de training diversifiées.

4. ANALYSE DES GAINS

4.1 Gain de temps par tâche

Tache	Classique	Claude Code	Ratio
Architecture 6 couches + modeles	2-3 sem.	1h	x80
74 tests unitaires	1 sem.	30 min	x56
Simulation 10k+ paiements	2 sem.	2h	x40
Streamlit 11 pages premium	3-4 sem.	4h	x30
Audit 22 bugs + perf 2.5x	2 sem.	1h	x56
ML pipeline C4 + training	2-3 sem.	2h	x40
522 verbatims 12 langues	1-2 sem.	1h	x40
Documentation (README x5, rapport)	1 sem.	30 min	x56

Total : 3-4 mois / 3-4 devs -> 6 jours / 1 personne

4.2 ROI operationnel estime

Indicateur	Sans IA	Avec le systeme
ETP reconciliation	~8 ETP	< 1 ETP
Temps par paiement	3-5 min	17 ms
Taux d'erreur	2-5%	< 0.1%
Delai de cloture	J+2 a J+5	Temps reel
Recommandation	Aucune	ML top-5 (77%)

4.3 Qualite du livrable

- 127 tests unitaires (100% pass), zero regression
- 22 bugs trouves et corriges par audit automatique
- Performance optimisee : 497 paiements/seconde (2.5x le baseline)
- 41 commits atomiques avec messages descriptifs
- 0 dette technique, 0 TODO, 0 hack

5. RECOMMANDATIONS

5.1 Prochaines étapes

- Connecter aux flux bancaires reels (SWIFT/SEPA) en remplacement de la simulation
- Activer la couche LLM (C5) avec l'API Claude pour traiter les 10% de cas restants
- Deployer une API REST (FastAPI) pour integration dans le SI existant
- Mettre en production avec monitoring MLOps et boucle de feedback C6->C4
- Etendre le perimetre : reconciliation inter-company, reconciliation bancaire

5.2 Investissement estime pour la mise en production

Phase	Duree	Ressources
Connexion donnees reelles	2-3 sem.	1 dev + 1 metier
Activation LLM (C5)	1 sem.	1 dev
API REST + integration SI	2-3 sem.	1 dev backend
Recette + mise en prod	2-3 sem.	1 dev + 1 ops
TOTAL	2-3 mois	2-3 personnes

5.3 Conclusion

Ce projet demontre qu'un agent IA comme Claude Code permet de diviser par 30 a 80 le temps de developpement d'un systeme complexe, tout en maintenant un niveau de qualite eleve (127 tests, audit automatique, zero dette technique).

Le systeme livre est fonctionnel, teste, documente, et pret pour une connexion aux donnees reelles. Le taux d'automatisation de 90%+ avec des recommandations ML a 77% de precision pour les cas restants represente un gain operationnel significatif pour l'activite de factoring.

Le pair-programming humain + IA est un multiplicateur de productivite sans precedent.

ANNEXE — Inventaire technique

Stack technique

Langage	Python 3.11
ML	LightGBM 4.6, XGBoost 3.2, scikit-learn 1.8
NLP	rapidfuzz 3.x, unidecode
Frontend	Streamlit 1.56, Plotly 6.6
LLM (optionnel)	Anthropic Claude API, OpenAI API
Tests	pytest 9.x (127 tests)

Arborescence du projet

reconciliation/	Coeur du systeme (11 modules)
c0_preprocessing.py	Normalisation (14 transforms)
c1_exact_matching.py	Matching deterministe (7 regles)
c2_business_rules.py	Regles metier (12 tolerances + subset sum)
c3_nlp_fuzzy.py	Fuzzy matching (8 algos + OCR fix)
c4_ml.py	ML ensemble (42 features + explainability)
c5_llm.py	LLM client (API key injection + budget)
c6_human_review.py	File de revue (heapq + feedback)
orchestrator.py	Pipeline end-to-end
debtor_profiler.py	Profiling comportemental par debiteur
utils.py	Utilitaires partages (normalize_ref, typo...)
tests/	127 tests (6 fichiers)
app/	Streamlit + HTML export + PDF
data/	522 verbatims (12 langues)
examples/	3 scripts de simulation

Metriques du projet

Lignes de code	14 548
Commits git	41+
Tests	127 (100% pass)
Verbatims	522 templates, 19 categories, 12 langues
Mutations typo	16 operateurs, 3 niveaux de severite
Bugs corriges	22 (audit automatique)
Gain perf	2.5x (200 -> 497 p/s)